

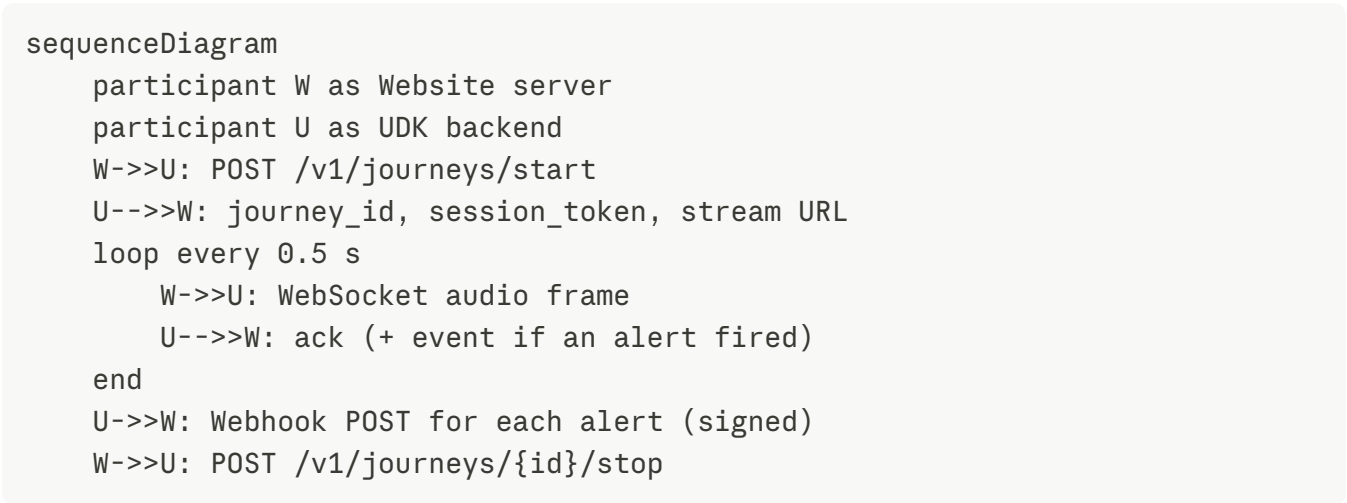
UDK Detection API: Integration Guide

Sep 26, 2026 · @Sohan

The website server streams live microphone audio to the UDK backend over a WebSocket; the backend returns an alert within seconds when it hears a danger phrase (UDK) or a sustained scream or cry (layer acoustic_scream, always TRIGGER_VERIFY).

Overview

One session (a "journey") covers one user's live audio from start to stop. The website server opens it, streams audio, and gets alerts back two ways: on the stream itself and by a signed webhook POST.



The WebSocket reply reaches whoever holds the stream; the webhook reaches the website server even if the stream holder changes, so use the webhook to drive the alarm and the 2-minute authorities timer.

Item	Value
Base URL	Given at deployment (HTTPS / WSS only)
Auth for HTTP routes	Header X-Service-Token: <token>
Auth for the stream	session_token query parameter from the start call
Audio	16 kHz, 16-bit signed little-endian PCM, mono

Item	Value
Time to alert	Target 5-10 s from the phrase being spoken

Step 1: Start a session

Call this once when a user starts being monitored. Calling it again with the same `journey_id` returns the same session, so a retry after a network error is safe.

```
POST /v1/journeys/start
X-Service-Token: <token>
Content-Type: application/json

{
  "user_id": "user-123",
  "journey_id": "optional-your-own-id",
  "personal_udk_phrase": "optional custom code phrase"
}
```

Field	Required	Meaning
<code>user_id</code>	Yes	The website's ID for the user; comes back on every alert
<code>journey_id</code>	No	Your own session ID; one is generated if left out
<code>personal_udk_phrase</code>	No	The user's custom UDK; rejected with 400 and a list of errors if it is too short or too common

Response:

```
{
  "journey_id": "3f2a9c1e-...",
  "session_token": "secret-for-this-session",
  "stream_endpoint": "/v1/journeys/3f2a9c1e-.../stream",
  "heartbeat_interval_s": 7
}
```

Keep `session_token` secret: it is the only credential the stream accepts.

Step 2: Stream audio

Open a WebSocket and send audio as binary frames, about one every 0.5 s. A wrong or expired token closes the socket with code 4401.

```
wss://<host>/v1/journeys/{journey_id}/stream?session_token=<session_token>
```

Binary frame format

Bytes	Content
0-7	<code>seq</code> : unsigned 64-bit, big-endian; 0 for the first frame, then +1 per frame
8-15	<code>client_ts</code> : 64-bit float, big-endian; Unix time the audio was captured
16-end	Audio: 16 kHz, 16-bit signed little-endian PCM, mono (0.5 s = 16,000 bytes)

- **Sequence numbers make retries safe.** A frame sent twice is ignored; frames arriving out of order are reordered. Never reuse a `seq` for different audio.
- **Every frame gets an ack:** `{"type": "ack", "ack_seq": 41, "server_ts": ...}`. `ack_seq` is the highest frame stored with nothing missing before it; after a reconnect, resend from `ack_seq + 1`.
- **Send a heartbeat every 7 s** as a text message: `{"type": "heartbeat"}`. With no frame or heartbeat for 20 s, the session is marked `TIMED_OUT`; after an alert has fired, that grace drops to 10 s and the status becomes `TIMED_OUT_AFTER_TRIGGER`.
- **Convert on your side** if the microphone gives 44.1/48 kHz or stereo: resample to 16 kHz mono before sending.

Example (Python)

```
import asyncio, json, struct, time
import websockets

HEADER = struct.Struct(">Qd") # seq, client_ts

async def stream(base_wss, journey_id, session_token, pcm_chunks):
    url = f"{base_wss}/v1/journeys/{journey_id}/stream?session_token={session_token}"
    async with websockets.connect(url) as ws:
        last_hb = time.time()
        for seq, pcm in enumerate(pcm_chunks): # each pcm = 0.5 s of
16 kHz int16 mono bytes
            await ws.send(HEADER.pack(seq, time.time()) + pcm)
            msg = json.loads(await ws.recv()) # ack; alert events
follow as extra messages
            if time.time() - last_hb > 7:
                await ws.send(json.dumps({"type": "heartbeat"}))
                last_hb = time.time()
```

In production, read replies in a separate task: after a frame, the server can send an ack plus one or more `{"type": "event", ...}` messages.

Step 3: Receive alerts

Every alert has two levels: `TRIGGER_ALL` for a strong match and `TRIGGER_VERIFY` for a weaker one. Both must reach the user; the level only tells the website how confident the backend is.

Event fields

```
{
  "event_id": "9c1f0e2a7b3d4c5e",
  "journey_id": "3f2a9c1e-...",
  "user_id": "user-123",
  "decision": "TRIGGER_ALL",
  "udk_id": "UDK_03",
  "udk_type": "GENERAL",
  "layer": "exact",
  "confidence": 0.97,
  "timestamp": 17900000000.5,
  "server_received_ts": 17900000004.1,
  "transcript": "get away from me",
  "audio_segment_id": "3f2a9c1e-...:12000-14500",
  "location": null
}
```

Field	Meaning
<code>event_id</code>	Stable ID; the same alert always has the same ID, so drop duplicates by it
<code>decision</code>	<code>TRIGGER_ALL</code> (strong) or <code>TRIGGER_VERIFY</code> (weaker)
<code>udk_id</code> , <code>udk_type</code>	Which phrase matched, and <code>GENERAL</code> (one of the 20 shared phrases) or <code>PERSONAL</code> (the user's custom UDK); both <code>null</code> for a scream alert
<code>layer</code>	Why it fired: <code>exact</code> , <code>fuzzy</code> , <code>semantic</code> , <code>kws</code> (matched on sound; switched off in the default deployment, so it does not appear) or <code>acoustic_scream</code> (scream or crying, no words)
<code>confidence</code>	0-1
<code>timestamp</code>	When the audio was spoken (Unix time); <code>server_received_ts</code> is when it was detected
<code>transcript</code>	What speech-to-text heard (for a scream alert, whatever was heard in that stretch, often empty). Sensitive: store encrypted

Field	Meaning
<code>audio_segment_id</code>	Which stretch of audio (ms from session start); the audio can be fetched for review
<code>location</code>	Always <code>null</code> today

Events also carry `processing_version` and `model_version` for audit.

Webhook delivery

Give us one HTTPS URL and a shared secret. Each alert is POSTed there as the JSON above with these headers:

Header	Value
<code>X-UDK-Event-Id</code>	The <code>event_id</code>
<code>X-UDK-Signature</code>	<code>sha256=</code> + HMAC-SHA256 of the raw body, keyed with the shared secret

- **Always verify the signature** before acting. A forged alert could lead to a false report to the authorities.
- **Reply with any 2xx** once the alert is stored. Any other reply, a timeout (5 s) or no reply means the alert is kept and sent again on the next heartbeat (about every 7 s), so the same `event_id` can arrive more than once.

```
import hashlib, hmac

def is_genuine(raw_body: bytes, signature_header: str, secret: str) -> bool:
    expected = "sha256=" + hmac.new(secret.encode(), raw_body,
    hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature_header)
```

Compute the HMAC on the raw bytes received, before parsing the JSON.

Step 4: Stop, status and history

All routes need the `X-Service-Token` header.

Route	Purpose	Response
POST /v1/journeys/{id}/stop	End monitoring; checks the last audio before closing. Body: {"reason": "user_stopped"}	{"journey_id", "status": "FINALIZING"}
GET /v1/journeys/{id}	Session status	status: ACTIVE, FINALIZING, COMPLETE, TIMED_OUT or TIMED_OUT_AFTER_TRIGGER, plus start_time, last_heartbeat_at
GET /v1/journeys/{id}/events	Every alert of the session, in order; use it to catch up after an outage	{"events": [...]}
GET /v1/journeys/{id}/audio-url? user_id=...&session_token=...	A link to the session audio for review, valid for 5 min	{"url", "expires_at"}

Stop the session when monitoring ends. A session with no audio or heartbeats for 20 s (10 s after an alert) counts as timed out, but the status only changes when a route is called: there is no background check and no webhook for it. To notice a session going dark, poll GET /v1/journeys/{id}.

Errors, limits and the website's side

Code	Where	Meaning
400	start	Custom UDK rejected; errors lists why (too short, too close to ordinary conversation)
401	HTTP routes	Missing or wrong X-Service-Token
403	audio-url	session_token does not match the session
404	any {id} route	Unknown session

Code	Where	Meaning
429	HTTP routes	Rate limit for the service token exceeded (10 calls/min unless the server sets UDK_RATE_LIMIT_PER_MIN; the deploy example uses 600); retry with backoff
4401	WebSocket close	Wrong <code>session_token</code> , unknown session, or the session is no longer <code>ACTIVE</code>

What the website must do on an alert

1. **Treat both levels as real.** On `TRIGGER_ALL` or `TRIGGER_VERIFY` , ring an alarm and ask "Are you safe?". It keeps going until the user stops it.
2. **No response for 2 minutes: report to the authorities.**
3. **Deduplicate by `event_id`** . The same alert can arrive on the stream, by webhook and on retry.
4. **Watch for a session going dark after an alert.** `TIMED_OUT_AFTER_TRIGGER` means the audio stopped after an alert fired. The backend does not push this, so poll `GET /v1/journeys/{id}` while an alert is open; how to handle it is the website's decision.

Still to agree before go-live

- ☐ Base URL, service token and webhook secret (exchanged outside this doc)
- ☐ The website's webhook URL
- ☐ The Hugging Face read token for the model repository (from Sohan, through a secure channel)
- ☐ Whether `user_id` may be sent to the backend, or a pseudonymous ID should be used instead

Deploying the backend (UDK team)

The backend ships as one Docker image on a GPU server, built from the private repository [sohans1092004/model](#). Only the 30 files the service needs reach the server and the image; recordings and evaluation code stay out.

Version to deploy: commit 0848231 or later (2026-09-26). It adds the intent model, a fine-tuned mDeBERTa that silences alerts for negated or quoted danger phrases such as "I don't need any help". The code is on GitHub; the model weights are on Hugging Face, so a

deployment needs access to both (steps 1 and 2).

What the intent model changes for the website

- **No API change.** Endpoints, events and fields stay the same.
- **Fewer alerts.** A standard-UDK alert is not sent when both speech-to-text engines heard the same words and the model is confident they are not a request for help ("I don't need any help", "the girl kept shouting stay away from me in the movie").
- **Never silenced:** custom code phrases (`personal_udk_phrase`), scream alerts (`acoustic_scream`), and any alert where the two engines heard different words.
- **Measured on a T4 GPU (2026-09-26):** speech false alarms on real conversation fell from 16 to about 4–5.5 per hour; 76 of 84 prepped danger runs are caught (the 3 lost runs are one garbled recording the old rules caught by chance). It adds about 28 ms per alert.
- **Switching it off** needs no rebuild: set `UDK_INTENT_GATE=` (empty) in `.env` and restart.

Server needs

Item	Requirement
GPU	NVIDIA; the evaluations ran on a T4 (16 GB)
Software	NVIDIA driver for CUDA 12, Docker 23 or later (BuildKit), NVIDIA Container Toolkit
Disk	About 16 GB for the image, plus the data volume
Network	HTTPS in front (nginx, Caddy or a cloud load balancer) that also carries WebSockets; the service itself listens on plain HTTP port 8000
GitHub access	A personal access token or deploy key, since the repository is private
Hugging Face access	A read token for the private model repository (step 2)

1. Fetch the code

```
git clone --filter=blob:none --no-checkout
https://github.com/sohans1092004/model.git udk
cd udk
git sparse-checkout init --no-cone
git sparse-checkout set /deploy/ /beats_vendor/ /kws_references.npz \
  /api.py /audit.py /beats_distress_detector.py /crypto.py /db.py
/db_postgres.py \
  /enrollment.py /event_delivery.py /intent_gate.py /kws.py /language_id.py
/metrics.py \
  /scream_trigger.py /semantic.py /separation.py /storage.py /storage_s3.py \
  /stt.py /stt_confidence_gate.py /udk_engine.py /udks.py /vad.py
/webhook_delivery.py
git checkout main
```

`--filter=blob:none` downloads only the checked-out files, so the audio in the repository history never reaches the server. Fetch on the server with git, not by copying a Windows folder: Windows line endings break the Dockerfile and `.env`.

2. Get access to the model weights

The fine-tuned model (1.1 GB) is in the private Hugging Face repository `sohans1092004/udk-intent-mdeberta`, not in GitHub.

1. Sohan creates a fine-grained **read** token limited to that one repository and hands it over through a secure channel (a password manager or your secrets store), never by chat or email.
2. On the build machine, keep it in an environment variable, never in a file inside the repository:

```
export HF_TOKEN=hf_... # the read token from Sohan
```

3. Optional check that the token works:

```
curl -s -H "Authorization: Bearer $HF_TOKEN"
https://huggingface.co/api/models/sohans1092004/udk-intent-mdeberta | head -c
120
```

JSON starting with `{"_id"` means access works; an error means the token is wrong or not scoped to this repository.

3. Build the image

```
DOCKER_BUILDKIT=1 docker build -f deploy/Dockerfile -t udk-api \
  --build-arg UDK_INTENT_MODEL_REPO=sohans1092004/udk-intent-mdeberta \
  --build-arg
UDK_INTENT_MODEL_REVISION=6131acc0f5c583d3ef24b7da1f293d207f26a08b \
  --secret id=hf_token,env=HF_TOKEN .
```

- Takes roughly 20–40 min and bakes about 5 GB of model weights into the image, so the service starts without downloads.
- The token is a build secret: it is used during the build and is not stored in the image.
- The revision pins the exact model the settings were calibrated on; change it only when Sohan publishes a new model.
- A successful build prints `intent model sohans1092004/udk-intent-mdeberta -> /app/models/intent_mdeberta`. Without the two `--build-arg` lines the image still builds, but runs without the intent model.

4. Configure

```
cp deploy/.env.example .env
```

Fill in `UDK_SERVICE_TOKEN`, `UDK_WEBHOOK_URL` and `UDK_WEBHOOK_SECRET`. Keep every comment on its own line: Docker treats anything after `=` as the value. The detection settings are already the validated configuration, including `UDK_INTENT_GATE=/app/models/intent_mdeberta` and `UDK_INTENT_THRESHOLD=0.02`; leave them as they are. A server set up before 2026-09-26 needs these two lines added to its `.env`.

5. Run

```
docker run -d --name udk --gpus all --restart unless-stopped \
  --env-file .env -p 8000:8000 -v udk-data:/data udk-api
```

6. Check it

```
docker logs udk 2>&1 | grep -iE "warn|error|cuda|intent gate"
curl -H "X-Service-Token: $TOKEN" http://localhost:8000/metrics
```

Log line	Meaning
<code>intent gate ON:</code> <code>/app/models/intent_mdeberta (threshold</code> <code>0.02, cuda)</code>	Everything is in place
<code>... intent gate OFF</code>	The image was built without the model: redo step 3 with the build arguments. The service still works, without the intent model
Speech-to-text fell back to CPU	The GPU is not being used

Then stream one test clip and confirm the alert reaches the website's webhook.

Updating

```
git sparse-checkout add /intent_gate.py # once, on servers set up before
2026-09-26
git pull
# then the build command from step 3, then:
docker rm -f udk && docker run -d --name udk --gpus all --restart unless-
stopped \
  --env-file .env -p 8000:8000 -v udk-data:/data udk-api
```

The sparse file list stays in place across pulls, so a server set up earlier does not receive new files on its own: without the `sparse-checkout add` line the new `api.py` fails to start. Live sessions in progress are lost on restart, so update when traffic is low.

Rules for running it

- **One worker, one container per GPU.** Live sessions are held in memory. More users means more containers behind a load balancer that keeps each `journey_id` on the same container.
- **Back up the `/data` volume, keys included.** It holds the encryption keys, signing secret, database, audio and the alert retry queue. Losing the keys makes stored audio and events unreadable.
- **The service refuses to start** with an empty or default service token, a non-HTTPS webhook URL, or a webhook without a secret.

- **Later option:** build the image in CI and push it to a container registry, so the server only pulls a finished image and never holds source code.